A Hardware Implementation for Deep Reinforcement Learning Machine

Pham Cong Thinh[1,2], Nguyen Tien Luan[1,2], Lam Duc Khai[1,2]
[1]University of Information Technology, Ho Chi Minh City, Vietnam
[2]Vietnam National University, Ho Chi Minh City, Vietnam
Corresponding Email: khaild@uit.edu.vn

2022 RIVF International Conference on Computing and Communication Technologies (RIVF) | 978-1-6654-6166-5/22/$31.00 ©2022 IEEE | DOI: 10.1109/RIVF55975.2022.10013789

***Abstract*—Deep Reinforcement Learning (DRL) algorithm is used in many areas of life where an agent learns how to interact with the environment to achieve a certain goal. The Deep Q-Network (DQN) has become a benchmark and building point for many DRL researchers. The DQN maps input states to (action, Q-value) pairs. In this paper, we propose a hardware architecture to implement the DQN algorithm, suitable for real-time applications. Its main features are low power and suitable for limited hardware resources. We have implemented the design on Xilinx Virtex 7 (XC7VX485tffg1927-1) with a Mountain Car simulation environment. Performance results are evaluated in terms of hardware resources, frequency, and power consumption with other reinforcement learning algorithms. Our work has been processed at 131MHz, the neural is 2x24x24x3 and the power consumption is 0.921 w.**

***Index Terms*—Deep Q-learning network, Deep Q network, DQN , Reinforcement Learning , FPGA, machine learning**

I. INTRODUCTION

Nowadays, machine learning is a trend in almost every field. Reinforcement learning (RL) is part of machine learning. It defines a random agent that must take appropriate actions to maximize the reward. Reinforcement learning differs from supervised learning in that in supervised learning, the training data is labeled with it so that the model is trained with the correct answer itself. While in reinforcement learning, there is no answer the reinforcers decide what to do to perform the given task. It through trial and error [2] interacts with its environment. Goals in RL are defined by functional rewards, which act as reinforcement or punishment that depend on one's performance towards the desired goal.

Reinforcement learning has been around for 30 years, however recent innovations in reinforcement learning have made some gains. One of the first achievements of Reinforcement Learning was the successful conquest of human games such as chess, Go, Starcraft 2 or Dota, opening a new era for the path of Artificial Intelligence development. Google DeepMind's AlphaGo is the result of combining deep ANN, supervised learning, Monte Carlo tree search and reinforcement learning to defeat the greatest Go player in history, Lee Sedol, by a score of 4-1.

Autonomous vehicles are also one of the prominent applications of reinforcement learning. Scientists say that reinforcement learning algorithms are the heart or soul of deep learning networks - when machines will learn by themselves through the process of implementation, iterative and continuous improvement. Some of the tasks to which autonomous driving can apply reinforcement learning include trajectory optimization, motion planning, propulsion...

Famous RL techniques like Q-learning [2] are often inflexible. It needs to create a Q-table which is a matrix whose dimension is [state, action]. So the Q-table cannot be used in environments where there is no certain number of states. Also Q-table is not very efficient with multi-state space and many actions because it requires a lot of memory to store Q-table.

Deep Reinforcement Learning (DQN) [4] [5] is a combination of reinforcement learning (RL) and deep learning and it is also the most trending machine learning field at the moment because it can solve a series of complex decision-making tasks where machines were previously incapable of solving real-world problems like human intelligence. The Deep Q Network (DQN) [5] algorithm is in fact much more stable by adding an experience replay (ER) and neural network (Neural Network) technique. It solves the Bellman equation by adjusting the parameters of the neural network towards the Bellman objective. Deep neural network is added to get high generalization ability and ER technique to reduce dependence on input data series.

Our proposed design is inspired by the DQN algorithm, which proposes a DQN model to solve the Mountain Car problem that can be applied on the FPGA platform with higher energy efficiency than based on the CPU. In our work, we improve frequencies, resources and power through hardware design with pipeline architecture.

The rest of the paper is organized as follows: In Section 2,we present the related knowledge about reinforcement learning and DQN . In Section 3, we introduce our DQN Core IP. In Section 4,We extensively evaluate our proposed design by comparison with DQN's on-chip and FPGA implementations. Finally, in Section 5, we conclude the paper.

Fig. 1 shows a deep reinforcement learning model,

Authorized licensed use limited to: Cyprus University of Technology. Downloaded on March 24,2025 at 13:35:18 UTC from IEEE Xplore. Restrictions apply.

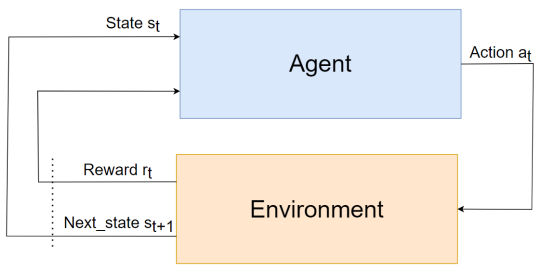

Fig. 1. Reinforcement learning framework

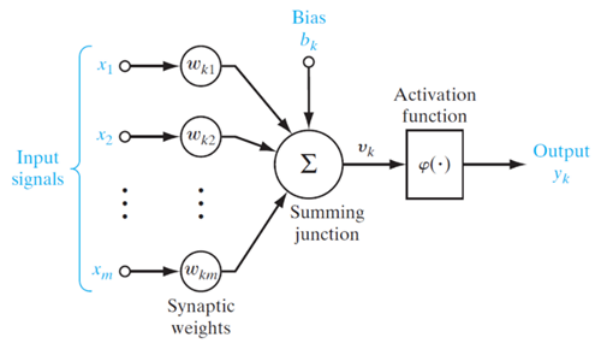

Fig. 2. Feed-forward ANN structure.

with signals interacting between the agent and the environment.

II. BACKGROUND

A. Q-LEARNING ALGORITHM

Q-learning [1] [2] is an off-policy reinforcement learning algorithm that seeks the best action to perform in the current state. It is considered off-policy because its learning function can be outside the current policy, such as performing random acts that do not require a policy. More specifically, Q-Learning seeks to learn a policy that maximizes the total reward received. The letter "Q" in Q-Learning is understood as "quality", it denotes the value of the reward brought when performing an action at a certain state - can simply be understood as a pair of states, action that correspond to a Q value.

The future rewards obtained through repeatedly choosing one of the actions based on existing Q values, performing a future action, and updating the Q-function in the current state. Assume that we choose a future optimal action based on the largest Q value. After choosing an optimal action, we move to a new state, receiving a reward in the process. We assume that the new state is s_{t+1}, Optimal value for the next state is found by iterating all Q values in the next state and finding an action a_{t+1} to generate the optimal value Q value in the state. Now that we have the optimal Q value for the next state, we update the current Q value [3] using the following Q update value function:

$$Q_{new}(S_t, A_t) \leftarrow Q_{old}(S_t, A_t) +$$
$$\alpha(R_{t+1} + \gamma * maxQ(S_{t+1}, a) - Q_{old}(S_t, A_t)) \quad (1)$$

B. Artificial Neural Networks

Artificial Neural Networks (ANN) [3] [6] are inspired by how the human brain works and consist of simple processing elements (PE, artificial neurons) that are interconnected by the weighted connections . Therefore, this neural network will change, they depend on the input and output.

ANN consists of 3 main components: Input layer and output layer consist of only 1 layer, hidden layer can have 1 or more layers depending on the specific problem. ANN works in a way that describes how the nervous system works with its interconnected neurons. In ANN, except for the input layer, all the nodes of the other layers are full-connected to the nodes of the previous layer. Each node of the hidden layer receives the input matrix from the previous layer and combines it with the weights to get the result. As shown in Fig. 2, a neuron forms the basis for designing the ANNs.

A neuron constructs the basis of the ANN design. There is an adder for summing the weighted inputs and a bias for increasing or lowering the net input of the activation function, respectively. The weighted sum of all the inputs is calculated using the Equation

$$\sigma = (\sum_{1}^{n} x_i w_i) \quad (2)$$

The output of a perceptron, also called the firing rate is calculated by passing the value of V through the activation function as follows:

$$o = f(\sigma) = \frac{1}{1 + e^{-\sigma}} \quad (3)$$

C. Deep Q-Learning Networks

Recently, a new RL [5] algorithm when combined with the Deep Q network has brought a lot of stability and overcomes the limitations of Q-learning. Like Q-learning, it solves the Bellman equation itself by adjusting the parameters of the Q lattice toward the Bellman objective. DQN, as shown in Fig 3, differs from Q-learning in two ways. First, DQN uses experience replay memory. Instead of each network state updating once, we store the states in memory. At each time step t during the agent interaction with the environment it stores the packet experience =(st, at, rt, st + 1) into the replay memory D. Then perform sampling into small batches and put them into the learning. This helps to diversify the input and avoid Neural overfitting. Second, DQN maintains two separate networks Q current Q(s,a;θ_i) and Q target Q(s,a;θ_i^-), with current parameters θ_i in Q current and old parameters θ_i^- in Q target. The Q current parameters can be updated multiple times per time step and copied to the Q target after N iterations. At every

Authorized licensed use limited to: Cyprus University of Technology. Downloaded on March 24,2025 at 13:35:18 UTC from IEEE Xplore. Restrictions apply.

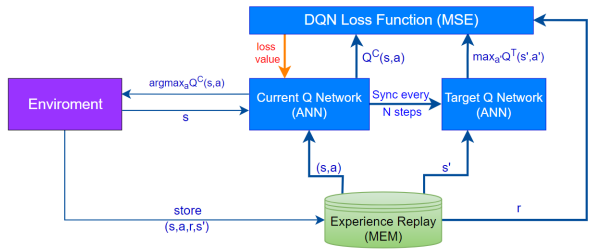

Fig. 3. The DQN algorithm

iterative update the current parameters Q are updated, to minimize the mean squared Bellman error for the old parameters the Q target function by optimizing the loss function. The purpose of the loss function is to force the Neural to learn how to estimate the Q-Value for actions correctly, so of course the Loss function must calculate the error between the actual and predicted Q-value. The loss function can be written as follows:

$$L_i(\theta_i) = E[(r - \gamma max_{\alpha'} Q(s', a'; \theta_i^-) - Q(s, a; \theta_i))^2] \tag{4}$$

Finally, actions are selected at each time-step t by an ϵ-greedy behavior with respect to the current Q-network Q(s, a; θ).

III. HARDWARE ARCHITECTURE

A. Overview of the design

Our design, the DQN Architecture is implemented based on the DQN architecture first proposed by Mnih [9]. We have changed the parameters appropriate for the MountainCar environment responsiveness. The DQN architecture is designed by pipelined verilog code and uses 32 bit floating-point numbers for computation.

The DQN architecture shown in Fig. 4 is composed of three main blocks: Artificial Neural Network (ANN), Epsilon Greedy (Policy), and Experience Replay Memory (ER). In addition, the ANN block is composed of four other blocks: Main Net, Target Net, Loss Function (LF), and Soft Update.

The DQN architecture receives the states, done and reward signal from the environmental observation, then the result is an interactive action. In Fig. 4, signals used in interactive mode are highlighted in blue, and in training mode are red.

Before starting the system, it is necessary to initialize them with an initial set of weights, which will be stored in the memory of the ANN. The DQN will use this set of weights for its first computations. After successfully initializing the weights, the system can operate in an interactive state.

In interactive mode, state values are fed simultaneously to the RM to store experience and the ANN block for computation, action values from the output of the ANN block are fed into the Policy block and interactive action selection is selected. under the policy of greed. This process is performed continuously until the memory capacity is enough to conduct the learning process.

In training mode, start from selecting a small sample of experience values stored in interactive mode inside the RM memory into the mini-batch, then put the values contained in the mini-batch into the ANN block to train their networks, helping to optimize the weights of this network. After giving all the values contained in the mini-batch, the agent will use the latest weights to return to interact with the environment.

The interaction and training will be performed continuously until the agent clearly understands its interactive environment, or the system's weight has been optimized. These values are then compared and evaluated optimal level of the system.

The algorithm for Deep Q-learning network with a learning rate of 0.001 and a discount factor of 0.8 which we use shown in algorithm 1.

Algorithm 1 DQN Algorithm

Initialize parameters $\theta 1$, using random values R $\in$ [-1, 1]
Initialize parameters $\theta 2 \leftarrow \theta 1$
Initialize $\epsilon, \epsilon decay, \epsilon min$
Initialize Experience Memory D, DSize, SizeTrain, MaxSize
Initialize $\alpha, min\alpha, \gamma$
Initialize BatchSize
for $episodes \in 1...$ **do**
  $S_t \leftarrow env.reset()$
  for $step \in 1...$ **do**
    // SELECT ACTION
    if $randomvalue < \epsilon$ **then**
      a $\leftarrow$ random_action_value
    else
      a $\leftarrow maxQ(S_t, a)$
    // Observe
    $(S_{t+1}, r, done_t) \leftarrow env.step(a)$
    pack $\leftarrow (S_t, a + S_{t+1}, r, done_t$
    Store(pack) $\rightarrow ExperienceMemoryD$
    if $done_t$ == 1 **then**
      if DSize SizeTrain **then**
        BatchSize $\leftarrow$ Retrieve $pack$ random from D
        for train $\in$ 1... miniBatch **do**
          Update $\theta 1$ with batch[train] with SGD
        $\theta 2 \leftarrow \theta 1$
      Break
// Finish Episode
$\epsilon = \epsilon * \epsilon decay$

B. Our Implementation

Authorized licensed use limited to: Cyprus University of Technology. Downloaded on March 24,2025 at 13:35:18 UTC from IEEE Xplore. Restrictions apply.

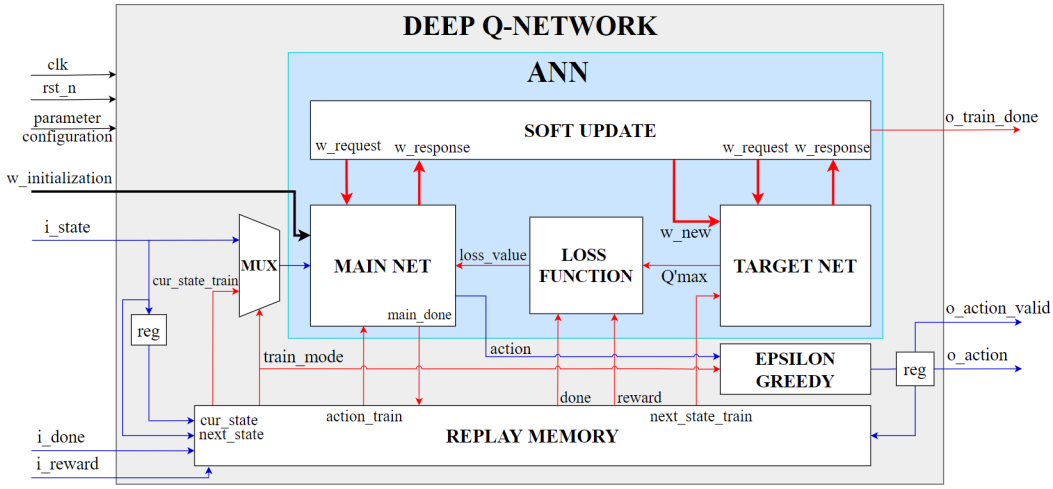

Fig. 4. Overview architecture of Deep Q-Network.

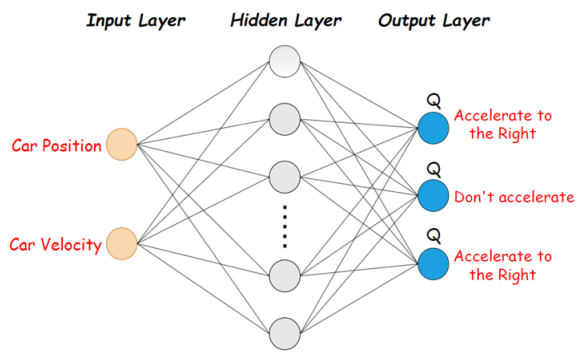

Fig. 5. ANN Model.

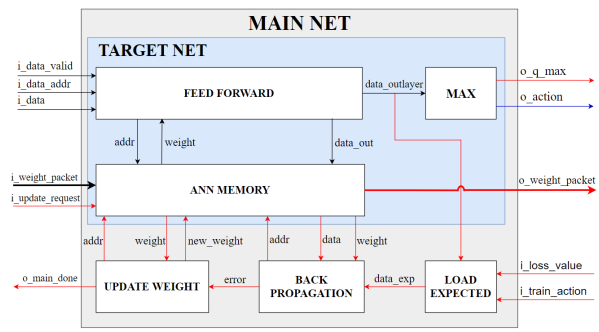

Fig. 6. The architecture of ANN Networks.

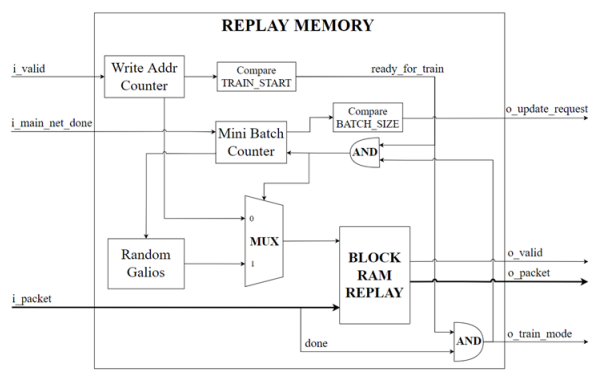

Fig. 7. The architecture of Replay Memory Block.

1) ANN Block: Store the values of the weights, get the state values as input, calculate and take the appropriate interactive action. They also update the weights themselves when in training mode. The ANN module shown in Fig. 4, belonging to the DQN architecture, includes two quite similar networks Main Net and Target Net designed based on the ANN network architecture proposed by [10].

The use of two ANNs makes the learning process optimized. Which, with the Main Net network, it can update the weights for itself, while the Target Net cannot do this, but waits for the update from the Main Net.

The architecture of these two networks is shown in Fig. 5 In general, the Target Net architecture is separated from the Main Net architecture. Only the characteristics of the block calculate Max. While the Main Net block requires action on the output, the Target Net block requires Q_{max} on the output. Fig. 6 shows the design of the architecture of these two networks.

2) Replay Memory Block: This block consists of the main memory stored as Block Ram, a memory element on the FPGA, this storage helps to store large amounts of data. Inside the Block Ram Replay block stores all input data, which are specific to the experiences the agent has experienced. Besides, Experience Replay Memory or Replay Memory shown in Fig. 7 works like a queue (FIFO), data is put into memory and replaces its oldest value when the memory is full. The address for storing these data is taken from the Write Addr Counter which always adds 1 when new data is entered.

Authorized licensed use limited to: Cyprus University of Technology. Downloaded on March 24,2025 at 13:35:18 UTC from IEEE Xplore. Restrictions apply.

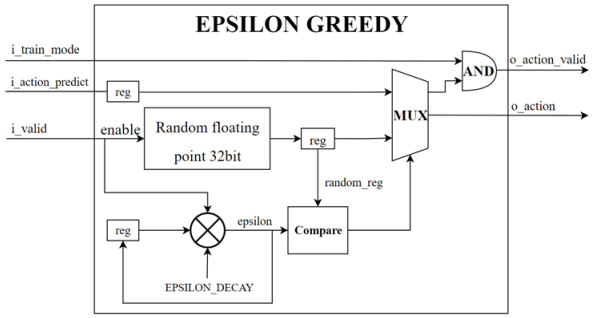

Fig. 8. The architecture of Epsilon Greedy Block.

3) Epsilon Greedy Block: This block is the decision-making policy description of the system designed based on the Epsilon-Greedy algorithm. The Epsilon Greedy block is shown on Fig. 8 has the function of choosing between mining and exploration at random. The purpose of this policy is to make the best reward for engaging in action.

IV. RESULTS

The Deep Q-Network architecture is made up of Verilog code and synthesized on the Vivado tool. The design is run post-timing implementation on Xilinx Virtex-7 board (XC7VX485tffg1927-1). The results presented in Table I show that the design does not exceed the allowable resources. Table II shows that our design operates at a frequency of 131.3 MHz and the power consumption is 0.921W that is much lower than that of conventional designs.

TABLE I
FPGA RESOURCE USAGE ON XILINX VIRTEX-7

Resource	Ultilization	Available	Ultilization %
LUT	85694	303600	28.23
LUTRAM	1581	130800	1.21
FF	142635	607200	23.49
BRAM	81.50	1030	7.91
DSP	1	2800	0.04
IO	152	600	25.33
BUFG	2	32	6.25

V. CONCLUSION

The project has completed the FPGA-based DQN system, capable of interacting with the environment and self-updating weights inside the ANN. However, the frequency target set out in the detailed outline has not been fully completed yet. The system of the implementation group only runs at 131 MHz, because the design of the group is quite large. each block inside is not optimized in terms of frequency as well as resources.

TABLE II
RESULTS OF OUR WORK COMPARISON WITH DIFFERENT ARCHITECTURES

	[7]	[8]	Our design
Platform	Alveo U50	VCU 1525	Virtex-7 VC707
Pipeline	Yes	Yes	Yes
Precision	Fixed 16-32b	Float 32b	Float 32b
Algorithm	DDPG	A3C	DQN
Clock	164 MHz	180 MHz	131.3 MHz
Power	15 W	18 W	0.921 W

ACKNOWLEDGEMENT

This research is funded by the University of Information Technology, Vietnam National University, Ho Chi Minh City.

REFERENCES

[1] L. M. D. Da Silva, M. F. Torquato and M. A. C. Fernandes, "Parallel implementation of reinforcement learning Q-learning technique for FPGA", IEEE Access, vol. 7, pp. 2782- 2798, 2018.
[2] S. Spanò et al., "An Efficient Hardware Implementation of Reinforcement Learning: The Q-Learning Algorithm," in IEEE Access, vol. 7, pp. 186340-186351, 2019, doi: 10.1109/ACCESS.2019.2961174.
[3] M.-J. Li, A.-H. Li, Y.-J. Huang and S.-I. Chu, "Implementation of deep reinforcement learning", Proc. 2nd Int. Conf. Inf. Sci. Syst., pp. 232-236, 2019.
[4] H. Watanabe, M. Tsukada and H. Matsutani, "An FPGA-Based On-Device Reinforcement Learning Approach using Online Sequential Learning," 2021 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW), 2021, pp. 96-103, doi: 10.1109/IPDPSW52791.2021.00022.
[5] Nair, A., Srinivasan, P., Blackwell, S., Alcicek, C., Fearon, R., De Maria, A., Panneershelvam, V., Suleyman, M., Beattie, C., Petersen, S. and Legg, S., Massively parallel methods for deep reinforcement learning. arXiv preprint arXiv:1507.04296, 2015.
[6] P. R. Gankidi and J. Thangavelautham, "FPGA architecture for deep learning and its application to planetary robotics," 2017 IEEE Aerospace Conference, pp. 1-9, 2017.
[7] J. Yang, S. Hong and J. -Y. Kim, "FIXAR: A Fixed-Point Deep Reinforcement Learning Platform with Quantization-Aware Training and Adaptive Parallelism," 2021 58th ACM/IEEE Design Automation Conference (DAC), 2021, (pp. 259-264).
[8] Cho, Hyungmin, Pyeongseok Oh, Jiyoung Park, Wookeun Jung, and Jaejin Lee. "Fa3c: Fpga-accelerated deep reinforcement learning." In Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems, pp. 499-513. 2019.
[9] V. Mnih, K. Kavukcuoglu, D. Silver, A.A. Rusu, J. Veness, M.G. Bellemare, A. Graves, M. Riedmiller et al.,"Human-level control through deep reinforcement learning", Nature, vol. 518, no. 7540, pp. 529-533, 2015.
[10] Abraham, A. (2005). Artificial Neural Networks. In Handbook of Measuring System Design (eds P.H. Sydenham and R. Thorn). https://doi.org/10.1002/0471497398.mm421.

Authorized licensed use limited to: Cyprus University of Technology. Downloaded on March 24,2025 at 13:35:18 UTC from IEEE Xplore. Restrictions apply.